Templating

Theme layouts are MiniJinja templates. The HTML layouts (`layouts/site.html` and `layouts/document.html`) and the paged layout (`layouts/pdf.typ`) all use the same engine, so the syntax on this page applies to both. The HTML templates and PDF templates pages document the values each layout receives.

Syntax

MiniJinja has two kinds of tags.

Interpolation prints a value:

```
<title>{{ doc.title }}</title>
```

Statements control flow. Loops repeat a block:

```
{% for file in css %}
<style>
{{ file.content }}
</style>
{% endfor %}
```

Conditionals show a block only when a value is set:

```
{% if site.logo %}
<img src="{{ site.logo }}" alt="{{ site.logo_alt }}">
{% endif %}
```

Includes pull in another template file, which is how themes share partials:

```
{% include "partials/site-footer.html" %}
```

The MiniJinja syntax reference covers the rest: filters, tests, macros, and more.

Context

Each layout receives a context: a set of named values you reference with `{{ }}`. The available names depend on the target.

- HTML layouts receive `site`, `css`, `js`, `doc`, `theme`, `target`, `store`, and more. See HTML templates.
- The paged layout receives `doc`, `theme`, `target`, and `store`. See PDF templates.

Both targets receive `theme`, `target`, and `store`.

Document store

Pass project-specific template values with `[store]` or `--set store.<name>=...`:

```
<title>{{ doc.title }}</title>
```

The completed document store is available as top-level `store` in both HTML and paged layouts. It includes project values, document initializers declared with `calepin.store.set()`, and values committed by successful `store-set` chunks. CLI `--set store.<name>=...` values take precedence over project and document initializers. In HTML templates, `store` sits at the top level, not under `site`:

```
<p>{{ store.course }}, {{ store.semester }}</p>
```

In a `layouts/pdf.typ` paged layout, read the same values and emit Typst:

```
#let course = "{{ store.course }}"
```